정비 전산 결제 연동 — 연동 방식 및 기술 정보

「정비 전산 결제 연동 개발 요청서」의 기술 자료입니다. 개발 기간 산정에 필요한 연동 구조, 토스 POS 플러그인 기술 정보, 결제 확인 방법, 통신 규약(권장안)을 정리했습니다.

1. 연동 구조

정비 전산 화면 → 정비 전산 서버 ⇄ 매장 토스 POS(결제 플러그인) → 결제

토스플레이스 Open API → 정비 전산 서버 (결제 승인·취소 알림)

- 매장 POS 의 플러그인이 정비 전산 서버로 **먼저 접속해 연결을 유지**합니다(POS → 서버 방향). 매장에는 공유기·방화벽 설정이나 추가 장비가 필요 없습니다.
- 정비 전산 서버에는 인터넷에서 접속할 수 있는 고정 주소(`https://`, `wss://`)가 필요합니다.
- 결제완료·취소는 두 경로로 받습니다. ① 플러그인이 보내는 결제 이벤트 ② 토스플레이스 Open API 결제 웹훅. **결제 화면이 열리면 플러그인 화면이 닫힐 수 있어 ②가 꼭 필요합니다.**
- 플러그인이 보낸 결제 이벤트는 금액·작업 번호를 그대로 믿지 않고, 결제 ID·주문 ID 로 토스 주문을 조회해 확인한 값으로 반영합니다. 그래야 매장 토큰만 가진 쪽이 결제완료를 만들어 낼 수 없습니다.

구성	기술	하는 일
결제 플러그인	토스 POS iframe 탭 앱, React + Vite + TypeScript, `@tossplace/pos-plugin-sdk`	서버 연결 유지, 장바구니 담기, 결제 화면 열기, 결제 이벤트 전송
정비 전산 서버	기존 서버에 기능 추가	POS 접속 창구(WebSocket + HTTP 폴링), 매장별 토큰, 담기 명령 보관·재전송, 결제 반영, 토스 웹훅 수신
정비 전산 화면	기존 화면에 기능 추가	[담기] 버튼, 상태 표시(결제대기·결제완료·취소(환불)), 다시 요청
토스플레이스 Open API 앱	토스 개발자센터 등록	결제 승인·취소 웹훅, 주문 조회

2. 처리 흐름

1. 직원이 정비 전산에서 [담기]를 누르면, 서버가 작업을 **결제대기**로 바꾸고 그 매장 플러그인으로 장바구니 명령(작업 ID, 품목·단가·수량)을 보냅니다. 플러그인이 꺼져 있으면 보관했다가 접속하면 보냅니다.
2. 플러그인이 POS 장바구니(DraftOrder)에 품목을 넣고, 장바구니를 다시 읽어 전산 내용과 같은지 확인한 뒤 결제 화면을 엽니다(`startPayment()`). 처리 결과(담김·실패)를 서버에 응답합니다.

3. 손님이 POS 에서 결제합니다.
4. 토스플레이스가 결제 승인 웹훅을 정비 전산 서버로 보냅니다. 서버는 주문 조회 API 로 그 주문 품목의 메모에서 작업 ID 를 찾아 **결제완료**로 바꿉니다. 플러그인 화면이 열려 있으면 플러그인도 결제 이벤트를 보내므로, 같은 결제가 두 번 올 수 있습니다(중복 제거 필요). 플러그인 이벤트도 토스 주문 조회로 확인한 뒤 반영합니다.
5. 결제를 취소하면 취소 웹훅이 와서 **취소(환불)**로 바꿉니다. 이 작업은 다시 [담기]할 수 있습니다.

3. 토스 POS 플러그인 기술 정보

3.1 개발·배포 환경

항목	내용
개발자센터	https://developers.tossplace.com (앱 등록, zip 업로드, 개발 배포, 검수, ACL 설정)
개발 문서	https://docs.tossplace.com (POS 플러그인: 임시 주문(DraftOrder)·HTTP·WebSocket·Payment·RateLimit·배포 가이드 / Open API: 웹훅·주문)
형태	iframe 탭 앱. POS 의 [기타 > 외부 서비스 설정]에 탭 하나로 나타납니다. 공식 템플릿 `iframe-plugin-ts`(React + Vite + TypeScript)
SDK	`@tossplace/pos-plugin-sdk` 0.0.31
배포물	zip 1 개. 루트에 `index.html`, `iframe-manifest.json`, `assets/` (Vite `base: './'`)
외부 통신 허용(ACL)	개발자센터 ACL 에 접속할 주소를 **프로토콜까지** 한 줄씩 등록합니다. 예: `https://<서버 도메인>`, `wss://<서버 도메인>`. 바꾼 뒤에는 POS 를 다시 시작해야 할 수 있습니다
시험	개발자센터 개발 배포 → 테스트 가맹점·테스트 단말에 설치
매장 설치	가맹점이 POS [설정 > 기타설정 > 서비스 연동]에 서비스 코드를 넣어 연결합니다(4 장의 Open API 앱도 같은 방법)
제약	**가맹점 한 곳에 POS 플러그인은 1 개만 설치할 수 있습니다**

3.2 사용하는 SDK 기능

SDK	용도
`draftOrder.get()`	지금 POS 장바구니 읽기(담기 전후 검증)

SDK	용도
`draftOrder.addLineItem(dto)`	장바구니에 품목 넣기(`key`, `item`, `itemPrice.priceValue`, `quantity`, `memo`)
`draftOrder.deleteLineItem(key)`, `clear()`	품목 빼기
`draftOrder.startPayment()`	결제 화면 열기
`draftOrder.on('update')`	장바구니 변경 감지
`payment.on('paid' / 'cancel')`	결제 완료·취소 이벤트(paymentId, orderId)
`order.getOrder(id)`, `order.getOrders({start, end})`	주문·결제 조회(놓친 결제 찾기)
`catalog.getCatalogs()`	POS 등록 상품 읽기(장바구니 품목의 바탕)
`merchant.getMerchant()`	토스 가맹점 ID(merchantId) 확인 → 서버에 알려 웹훅과 매장을 연결
`websocket.create(url, headers)`	서버와 실시간 연결(헤더로 인증 토큰 전달)
`http`	HTTP 호출(WebSocket 이 안 될 때 폴링)
`storage`, `secureStore`	처리 기록 저장, 토큰 보관

3.3 주의 사항

개발 기간 산정에 영향이 큰 부분입니다.

1. **결제 화면이 열리면 플러그인 화면이 가려지거나 다시 로드될 수 있습니다.** 탭 앱은 화면을 떠나면 닫힙니다. - 결제 결과는 웹훅(4 장)으로 받아야 합니다. - `startPayment()`를 부르기 **전에** 결제 대기 정보를 `storage` 에 저장하고, 화면이 다시 열리면 `order.getOrders()`로 놓친 결제를 찾습니다. - 처리한 명령 기록도 결제 화면을 열기 **전에** 저장합니다. 그래야 화면이 다시 로드된 뒤 서버가 같은 명령을 다시 보내도 또 담고 결제 화면을 또 여는 일(이중 결제 위험)이 없습니다. - 플러그인 화면이 열려 있어야 [담기]가 장바구니에 들어옵니다. 닫혀 있는 동안의 [담기]는 서버가 보관했다가 다시 열리면 보냅니다.
2. **`addLineItem` 의 `item` 을 최소 필드로 만들면 호출은 성공하지만 POS 주문 화면이 깨집니다.** POS 카탈로그에 등록된 상품 하나를 그대로 복사해 바탕으로 쓰고, 이름(`title`)·단가(`priceValue`)·면세

여부(`isTaxFree`)만 바꿔 넣습니다. 값이 null 인 `titleI18n`·`descriptionI18n` 은 지웁니다(SDK 가 null 을 거부). POS 에 상품이 하나 이상 등록되어 있어야 합니다.

3. **POS 는 같은 `key` 의 품목을 여러 줄 허용합니다.** key 를 다시 쓰면 장바구니에 중복이 쌓입니다(예: 결제 취소 뒤 다시 담을 때). 줄마다 새 key 를 쓰고, 결제 화면을 열기 전에 `get()`으로 다시 읽어 전산 내용과 정확히 같은지 확인합니다. 다르면 결제 화면을 열지 않고 실패로 응답합니다.
4. **품목 `memo` 에 작업 번호를 넣어 둡니다.** 토스 주문 정보의 품목(`lineItems[].memo`)에 함께 저장되므로, 웹훅 결제와 정비 작업을 잇는 연결 고리로 씁니다(예: `#1234 엔진오일 교환`). memo 는 POS 주문 화면에도 보입니다. 플러그인도 결제 이벤트를 보낼 때 그 순간의 장바구니가 아니라 **결제된 주문(`order.getOrder`)의 memo** 로 작업을 찾습니다. 장바구니 기준으로 정하면 다른 주문의 결제가 엉뚱한 작업에 붙을 수 있습니다.
5. **요청 한도**: `order.*` 등 일부 기능은 60 초 100 회·30 분 200 회 한도가 있습니다(합산). `draftOrder.*`는 한도 대상이 아닙니다. 초과하면 오류가 나므로 잠시 뒤 다시 시도합니다.

4. 결제 확인: 토스플레이스 Open API

항목	내용
앱 등록	개발자센터 → 내 애플리케이션 → Open API 앱 → 웹훅에서 수신 주소, 이벤트(결제 승인·취소), 서명용 시크릿을 등록합니다
매장 연결	가맹점이 POS [설정 > 기타설정 > 서비스 연동]에 Open API 앱의 서비스 코드를 넣어야 그 매장의 웹훅이 옵니다
수신 주소	앱 하나에 1 개. 모든 매장의 웹훅이 같은 주소로 오고 `merchantId` 로 매장을 구분합니다
이벤트	`payment.payment.approved.v1`(승인), `payment.payment.cancelled.v1`(취소)
본문	`{id, type, createdAt, merchantId, app, data: {payment: {id, orderId, amount, merchantId, state, cancelledAt}}}`
서명 검증	헤더 `x-toss-webhook-id`(중복 제거 키), `x-toss-timestamp`(밀리초), `x-toss-signature` = `v1=` + hex(HMAC-SHA256(웹훅 시크릿, `{timestamp}.{원본 본문}`))
전달 방식	최소 1 회 전달(같은 이벤트가 다시 올 수 있음). 2xx 응답이면 성공. 송신 IP `15.165.6.198`
주문 조회	`GET https://open-api.tossplace.com/api-public/openapi/v1/merchants/{merchantId}/order/orders/{orderId}`, 헤더 `x-access-key`, `x-secret-key`. 응답의 `lineItems[].memo`, `chargePrice.totalAmount`, `payments[]`를 씁니다

항목	내용
요청 한도	매장마다 초당 10 회(최대 100 회 누적). 넘으면 429

처리 방법

- 웹훅 본문에는 품목 정보가 없습니다. 그래서 받은 뒤 주문 조회 API 를 한 번 더 불러야 합니다.
- **승인 웹훅**: 주문 조회 → `lineItems[].memo` 에서 작업 ID 를 찾음(예: `^#(\d+)(\s|$)`) → 취소되지 않은 `payments[]` 금액 합이 `chargePrice.totalAmount` 와 같고, [담기]한 금액과도 같으면 **결제완료**. 금액이 다르면 직원이 확인하도록 표시합니다.
- 결제 직후 주문 정보가 늦게 반영될 경우에 대비해, 메모가 없으면 30 초 뒤 한 번 더 조회합니다. 그래도 없으면 정비 전산과 관계없는 결제이므로 무시합니다.
- **취소 웹훅**: 주문을 다시 조회해 남은 결제 금액이 0 이면 그 작업을 **취소(환불)**로 바꿉니다. 일부만 취소된 경우는 직원이 확인하도록 표시합니다.
- **매장 찾기**: 플러그인이 접속할 때 `merchant.getMerchant()`의 가맹점 ID 를 서버에 알려 주면, 서버는 매장 토큰과 merchantId 를 연결해 저장합니다. 웹훅의 `merchantId` 로 어느 매장인지 찾습니다.
- 웹훅은 다시 오거나 순서가 바뀌어 올 수 있습니다. `x-toss-webhook-id` 로 중복을 지우고, 상태는 웹훅이 도착한 순서가 아니라 **주문 조회로 확인한 실제 결제 상태**로 정합니다. 그래야 취소가 먼저 오고 승인이 늦게 와도 결제완료로 잘못 바뀌지 않습니다.

5. 정비 전산 서버 ⇄ 플러그인 통신 (권장안)

플러그인과 서버를 모두 개발사에서 만드시므로 통신 방식은 자유롭게 정하셔도 됩니다. 아래는 권장하는 방식입니다.

5.1 연결

항목	내용
경로	실시간: `wss://<서버>/pos/ws`. WebSocket 이 안 될 때: `GET /pos/poll`(롱폴링, 최대 25 초 대기) + `POST /pos/events`
인증	모든 요청과 WebSocket 연결에 `Authorization: Bearer <매장 토큰>`. 매장마다 다른 토큰(96 비트 이상 난수)을 발급하고, 직원이 플러그인 설정 화면에 넣습니다. 틀리면 401
연결 유지	플러그인이 접속해 `hello` → 서버가 `welcome`. 서버는 25 초마다 `ping`, 플러그인은 `pong`. 70 초 동안 메시지가 없으면 다시 접속(2 초에서 60 초까지 간격을 늘림)

항목	내용
자동 전환	WebSocket 이 3 번 연속 실패하면 HTTP 폴링으로 바꾸고, 10 분 뒤 WebSocket 을 다시 시도
POS 여러 대	매장마다 명령은 활성 단말 1 대에만 보냅니다
접속 시간	플러그인은 POS 에서 그 화면이 열려 있는 동안만 접속합니다(3.3 의 1 번)

5.2 메시지 (JSON)

방향	메시지	주요 필드
서버 → 플러그인	`welcome`	연결 확인
	`command` (`SYNC_CART`)	`workOrderId`, `label`, `lines[]`(`key`, `title`, `unitPrice`, `quantity`, `taxFree`, `productCode`), `startPayment`, `exclusive`(기본 true: 다른 작업의 정비 품목은 빼고 이 작업만 남김). 증분이 아니라 **그 작업 장바구니의 최종 상태**이며, `lines` 가 비어 있으면 POS 에서 뺍니다
	`ping`	연결 확인
플러그인 → 서버	`hello`	`clientId`, `pluginVersion`, `transport`, `processedIds`(처리한 명령 id), `merchantId`
	`ack`	`commandId`, `ok`, `error`(실패 사유). POS 장바구니에 반영됐다는 뜻이며 결제완료가 아닙니다
	`payment.paid`	`paymentId`, `orderId`, `workOrderIds`, `fullyPaid`, `paidValue`(주문 전체 누적 결제액), `unpaidValue`(남은 금액, 주문을 조회하지 못하면 -1)
	`payment.cancelled`	`paymentId`, `orderId`

5.3 처리 규칙

- **품목 key**: `gm-{작업 ID}-{라인 ID}`. 수량이 바뀌어도 그대로 둡니다. 플러그인은 POS 에 넣을 때 `_` + 영숫자 8 자를 붙여 매번 새 key 로 넣고(3.3 의 3 번), 서버에 보고할 때는 원래 key 로 되돌립니다.

- **명령 중복 방지**: 명령 id 는 영구히 고유하게 만듭니다. 플러그인은 처리한 id 를 기억해 같은 명령을 두 번 처리하지 않습니다.
- **명령 보관·재전송**: 서버는 `ack` 가 30 초 안에 오지 않으면 다시 보내고, 30 분이 지나면 만료합니다. 플러그인이 닫혀 있는 동안의 명령은 보관했다가 접속하면 보냅니다. 같은 작업에 새 명령이 생기면 아직 안 보낸 이전 명령은 버립니다.
- **결제 중복 방지**: 플러그인은 다시 접속할 때마다 최근 결제 이벤트를 다시 보내고, 웹훅으로도 같은 결제가 옵니다. 서버는 (종류, 결제 ID)로 한 번만 반영하고, **이미 결제완료인 작업에 다시 온 결제완료는 무시**합니다. 다만 플러그인 보고를 먼저 받았더라도 토스로 확인한 결과가 한 번은 바로잡을 수 있게 합니다. 이미 취소가 처리된 결제에 뒤늦게 온 결제완료는 반영하지 않습니다. 취소는 결제 ID 로 못 찾으면 주문 ID 로 찾습니다.
- **자동 결제완료 조건**: 토스 주문 조회로 확인한 결과 전액 결제이고, 결제 금액이 [담기] 금액과 같을 때만. `workOrderIds` 가 비어 있으면(직원이 POS 에서 직접 담은 상품만 결제) 기록만 합니다.
- **화면 상태와의 관계**: [담기] → **결제대기**, `payment.paid`(또는 승인 웹훅) → **결제완료**, `payment.cancelled`(또는 취소 웹훅) → **취소(환불)**. `ack` 가 실패하거나 명령이 만료되면 사유를 화면에 보여 주고 결제대기로 둬서 다시 [담기]할 수 있게 합니다. 금액이 다르거나 결제를 확인하지 못한 경우는 별도 상태를 만들지 않고 결제대기에 "확인 필요" 표시를 붙여 직원이 확인하게 합니다.

6. 개발 범위 (산정용 작업 목록)

구분	작업
결제 플러그인	① 설정 화면(서버 주소·매장 토큰) ② 서버 연결(WebSocket, 안 되면 HTTP 폴링, 끊기면 재연결) ③ 장바구니 명령 처리(카탈로그 상품 바탕 품목 생성, 새 key, 결제 전 검증) ④ 결제 화면 열기 ⑤ 결제 이벤트 전송·재전송 ⑥ 다시 열릴 때 놓친 결제 찾기 ⑦ 결과 응답(담김·실패 사유)
정비 전산 서버	① POS 접속 창구(WebSocket + HTTP 폴링) ② 매장별 토큰 발급·인증 ③ 담기 명령 보관·재전송·응답 확인 ④ 결제 이벤트 반영·중복 제거 ⑤ 토스 웹훅 수신·서명 검증·주문 조회·작업 매칭 ⑥ merchantId ↔ 매장 연결
정비 전산 화면	① [담기] 버튼 ② 상태 표시(결제대기·결제완료·취소(환불)) ③ 결제대기·취소(환불) 작업 다시 요청, 결제완료 작업은 막기
전달·시험 참여	아래 "시험을 위해 주실 것" 전달, 테스트 단말 시험 참여

토스 개발자센터 등록(POS 플러그인·Open API 앱), ACL·웹훅 설정, 개발 배포와 테스트, 검수·배포는 **저희가 진행합니다.** 웹훅 수신 주소는 정비 전산 서버로 등록하고, 서버에 필요한 웹훅 시크릿과 Open API 키는 저희가 전달드립니다.

시험을 위해 주실 것

1. 플러그인 소스와 빌드한 zip
2. 시험용 서버 주소(https). 시험 기간 동안 계속 켜 두어 주십시오
3. 시험 매장 토큰
4. 시험용 정비 전산 화면 계정([담기]와 상태 변화를 저희가 직접 확인)
5. 저희가 드리는 웹훅 시크릿과 Open API 키를 시험 서버에 넣어 주십시오